

Propuesta de mejora arquitectónica y plan de evolución

Sistema EIEInfo - Entrega 3

Raúl Villalobos Vega

IE-0417 - Diseño de Software

Junio 2026

Pregunta guía

¿Cómo puede evolucionar EIEInfo para reducir sus riesgos principales sin romper su funcionamiento actual?

- Se parte de los hallazgos de la auditoría profunda.
- Se propone una evolución incremental, no una reescritura.
- La meta es reducir acoplamiento, mejorar mantenibilidad y proteger cambios futuros.

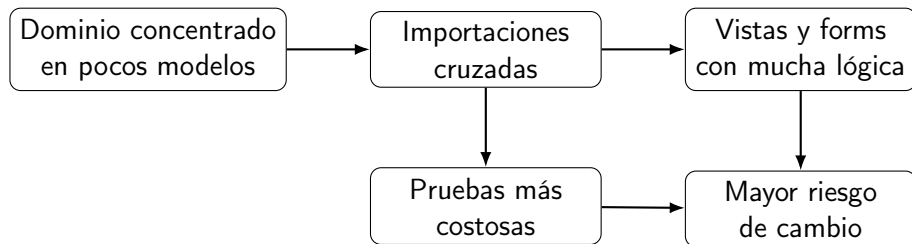
Decisión principal

No se recomienda migrar EIEInfo a microservicios.

Propuesta

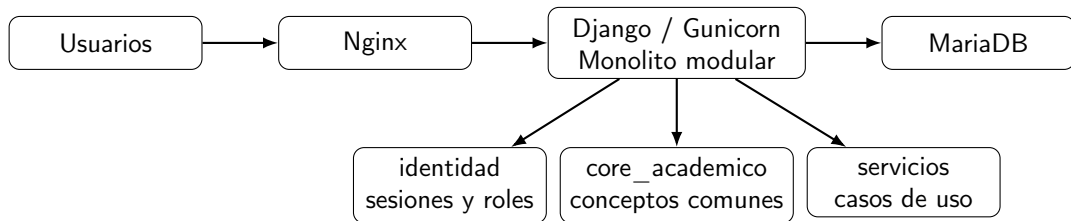
Mantener un monolito Django, pero convertirlo gradualmente en un monolito modular con fronteras internas más claras.

Diagnóstico que guía el plan



- `administrativos/models.py` concentra conceptos transversales: Ciclo, Lugar, Horario, Funcionario, Nombramiento.
- `profesores/views/consejo_asesor.py` tiene 1810 líneas.
- Consejo Asesor importa modelos, formularios y reportes de varias apps.
- La autenticación combina `django.contrib.auth` con sesiones propias.
- La configuración depende de archivos de secretos y del hostname `faraday`.

Arquitectura objetivo



La arquitectura externa se mantiene; la mejora ocurre dentro del monolito.

- **Incrementalidad:** no reescribir todo.
- **Compatibilidad hacia atrás:** mantener rutas y comportamiento visible.
- **Servicios antes que frameworks:** ordenar reglas internas sin cambiar tecnología.
- **Configuración externa:** sacar valores sensibles del código versionado.
- **Pruebas antes de refactorizar:** proteger el comportamiento actual.

Problema

`consejo_asesor.py` concentra demasiada lógica de cursos, horarios, aulas, nombramientos, asistencias, proyectos y reportes.

- Primero: pruebas de caracterización para capturar comportamiento actual.
- Luego: extraer servicios por flujo.
- Meta: que la vista coordine HTTP, pero no contenga toda la regla de negocio.

```
# Idea: la vista coordina, el servicio decide
resultado = horarios.validar_horario_grupo(grupo, horario)
if not resultado.es_valido:
    messages.error(request, resultado.mensaje)
```


Servicio	Responsabilidad
horarios.py	cupos, aulas, colisiones
nombramientos.py	crear, editar, limpiar nombramientos
asistencias.py	asignar horas y revisar estados
reportes.py	preparar datos del ciclo

La extracción debe hacerse por bloques, no toda de una vez.

Problema

Muchos módulos dependen directamente de `administrativos.models` para conceptos que no son solo administrativos.

- Crear una fachada o módulo `core_academico`.
- Iniciar con `Ciclo`, `Lugar`, `Horario` y validaciones reutilizables.
- No mover modelos ni tablas al inicio.
- Objetivo: reducir dependencias directas y riesgo de migraciones.

```
# Antes
from administrativos.models import Ciclo, Horario, Lugar

# Despues
from core_academico.services import obtener_ciclo_actual
from core_academico.horarios import validar_colision_horaria
```

Intervención P2: configuración y secretos

Matiz

No se asume explotación actual ni credenciales críticas confirmadas.

- Es una mejora preventiva de seguridad y portabilidad.
- Crear `.env.example` como plantilla sin valores reales.
- Leer `SECRET_KEY`, base de datos, correo y hosts desde variables de entorno.
- Evitar que producción dependa solo del hostname `faraday`.

```
SECRET_KEY = os.environ["DJANGO_SECRET_KEY"]  
DEBUG = os.environ.get("DJANGO_DEBUG") == "true"  
ALLOWED_HOSTS = os.environ["DJANGO_ALLOWED_HOSTS"].split(",")
```

Qué significa

identidad sería un módulo interno para centralizar usuario actual, sesión, roles y permisos.

- Profesores y administrativos usan `django_login`.
- Estudiantes usan sesión manual con llaves como `est_id`.
- La propuesta inicial no obliga a migrar todo de golpe.
- Puede empezar como una fachada sobre los mecanismos actuales.

```
usuario = identidad.obtener_usuario_actual(request)
if identidad.tiene_rol(usuario, "profesor"):
    return vista_profesor(request)
```

Aclaración de dominio

TFG y proyecto eléctrico son requisitos distintos para graduarse.

- No se propone fusionarlos.
- Se propone documentar qué pertenece a cada proceso.
- También conviene identificar conceptos compartidos: estudiante, profesor, ciclo, documentos y estados.
- Beneficio: menos ambigüedad ante cambios curriculares.

Intervención P4: observabilidad mínima

- Agregar una ruta /health/ para saber si Django y la base de datos responden.
- Registrar errores con contexto: módulo, operación, usuario y causa.
- Registrar inicio, éxito y fallo de tareas automáticas o cronjobs.
- No cambia la funcionalidad visible; mejora diagnóstico y operación.

Ejemplo

```
ERROR consejo_asesor.editar_grupo ciclo=2026-I grupo=45 error=colisión de horario"
```

```
def health(request):  
    with connection.cursor() as cursor:  
        cursor.execute("SELECT 1")  
    return JsonResponse({"status": "ok", "database": "ok"})
```

Prioridad	Intervención	Razón
P1	Consejo Asesor	principal riesgo de cambio
P1	Pruebas de caracterización	proteger comportamiento actual
P2	Dominio académico	reducir dependencia central
P2	Configuración externa	prevención y portabilidad
P3	Identidad	requiere migración cuidadosa
P3	Requisitos de graduación	clarificar dominio
P4	Observabilidad	operación sin alterar reglas

- 1 **Fase 0:** línea base y pruebas iniciales.
- 2 **Fase 1:** configuración externa, documentación y healthcheck.
- 3 **Fase 2:** fachada de dominio académico.
- 4 **Fase 3:** extracción gradual de servicios de Consejo Asesor.
- 5 **Fase 4:** identidad común y documentación de requisitos de graduación.

Riesgo	Impacto	Mitigación
Romper Consejo Asesor	Alto	pruebas y refactor por bloques
Migraciones innecesarias	Alto	fachadas antes de mover modelos
Configuración confusa	Medio	<code>.env.example</code> y guía
Duplicar servicios	Medio	documentar transición
Reglas históricas ocultas	Medio	pruebas y consulta funcional

- Nuevas reglas de horarios se prueban sin ejecutar toda la vista.
- administrativos deja de ser el único acceso a conceptos compartidos.
- La configuración no depende únicamente de faraday.
- No se versionan secretos reales.
- TFG y proyecto eléctrico quedan documentados como requisitos distintos.
- La documentación técnica refleja el estado real del sistema.

Idea final

La mejora más importante no es cambiar de tecnología, sino ordenar las fronteras internas del sistema.

- EIEInfo puede seguir siendo un monolito.
- Pero debe ser un monolito más modular.
- La evolución debe ser gradual, probada y compatible con lo existente.